

Programación y automatización en $\text{\LaTeX}$

Macros, lógica, bucles y generación de contenido

Rodrigo Martín Vidal Galán

Junio de 2026

$\text{\LaTeX}$ no es solo un procesador de texto: es un **lenguaje de programación** (de hecho, Turing-completo). Eso permite definir tus propios comandos, automatizar lo repetitivo, tomar decisiones lógicas, repetir con bucles e incluso **inyectar** resultados de otros lenguajes. Acá va lo esencial, de lo cotidiano (macros) a lo avanzado (`expl3`, Lua, generación).

Índice

1. Macros: tus propios comandos	3
2. Contadores y longitudes	3
3. Lógica: condicionales	3
4. Bucles	4
5. <code>expl3</code> : la capa de programación moderna (LaTeX3)	4
6. Inyectar otros lenguajes	4
7. Compilación condicional	5
8. Algoritmos y pseudocódigo	5
9. Buenas prácticas	5
Ver también	5

1. Macros: tus propios comandos

La herramienta más usada. `\newcommand` define un comando nuevo; si ya existe, $\text{\LaTeX}$ avisa (a diferencia de `\def`, que pisa en silencio):

```
\newcommand{\saludo}{Hola desde una macro!} % sin argumentos
\newcommand{\cuadrado}[1]{#1^2} % 1 argumento: #1
\newcommand{\vect}[1][v]{\mathbf{#1}} % argumento OPCIONAL (default v)
```

produce: ¡Hola desde una macro! x^2 , $(a+b)^2$, $\mathbf{v}$ y $\mathbf{w}$.

- `[n]` declara **n** argumentos, accesibles como `#1...#9`.
- `[n][def]` hace que el **primer** argumento sea opcional, con valor por defecto `def`.
- `\renewcommand` **redefine** uno existente; `\providecommand` lo define solo si no existe.

Por qué importa: una macro centraliza una notación. Si definís `\newcommand{\R}{\mathbb{R}}`, escribís `\R` en todos lados y, si algún día querés otro estilo, lo cambiás *en un solo lugar* (principio DRY).

Entornos propios. `\newenvironment` hace lo mismo para pares `begin/end`:

```
\newenvironment{remarca}{\par\noindent\textbf{Nota:~}\itshape}{\par}
```

2. Contadores y longitudes

$\text{\LaTeX}$ cuenta con **contadores** (enteros) y mide con **longitudes**. Podés crear y manipular los tuyos:

```
\newcounter{ejemplo}
\stepcounter{ejemplo} % +1
\setcounter{ejemplo}{5} % asignar
\arabic{ejemplo} / \roman{...} % mostrarlo (1.. / i..)
\the\value{ejemplo} % su valor

\newlength{\misangria}\setlength{\misangria}{1.5em}
\addtolength{\misangria}{2pt}
```

produce: el contador vale 3 (en romano: III).

Los contadores son la base de la numeración automática (secciones, ecuaciones, teoremas): todos son contadores que $\text{\LaTeX}$ incrementa solo.

3. Lógica: condicionales

Para que el documento **decida**, hay varias vías. La más legible es `\ifthenelse` (paquete `ifthen`):

```
\ifthenelse{\value{ejemplo} > 3}{es grande}{es chico}
\ifthenelse{equal{\modo}{borrador}}{...}{...}
```

produce: el contador 3 es chico.

Con `etoolbox` se trabaja con **interruptores** (*toggles*), muy cómodos para activar/desactivar partes:

```
\newtoggle{soluciones}
\toggletrue{soluciones} % o \togglefalse
\iftoggle{soluciones}{\textbf{Solucion:} ...}{}
```

También existen los condicionales primitivos `\ifnum`, `\ifdim`, y `\ifdefined` para comprobar si un comando existe.

4. Bucles

Para repetir, lo más práctico es `\foreach` (de `pgffor`, que trae `tikz`): itera sobre una lista o un rango:

```
\foreach \n in {1,2,3,4,5}{\n\ } % lista explícita
\foreach \n in {1,...,10}{\the\numexpr\n*\n\ $ } % rango
```

produce: $1^2 = 1$ $2^2 = 4$ $3^2 = 9$ $4^2 = 16$ $5^2 = 25$ $6^2 = 36$

`\foreach` es ideal para generar tablas, grillas o figuras repetitivas sin copiar y pegar. También existen `\loop... \repeat` (TeX) y los bucles de `expl3`.

Ejemplo: una fila de puntos con `\foreach`.



5. `expl3`: la capa de programación moderna (LaTeX3)

La programación «clásica» de TeX es poderosa pero críptica. `expl3` (ya parte del L^AT_EX actual) ofrece una sintaxis **ordenada**, con funciones y tipos de datos (listas, secuencias, *property lists*), pensada para escribir paquetes:

```
\ExplSyntaxOn
\cs_new:Npn \mi_doble:n #1 { \int_eval:n { 2 * #1 } }
\ExplSyntaxOff
```

Convenciones: los nombres llevan `_` y un sufijo de tipo `(:n, :N...)`, y dentro de `\ExplSyntaxOn` el espacio **no cuenta** (se usa `~` para espacio explícito), lo que hace el código legible. Es el estándar para desarrollo serio de paquetes; para el día a día alcanza con `\newcommand`.

6. Inyectar otros lenguajes

Lua (con LuaLaTeX). El motor `lualatex` incluye el lenguaje **Lua**; podés ejecutar código durante la compilación:

```
\directlua{ tex.print(2^10) } % imprime 1024
% o el entorno del paquete luacode para bloques mas largos
```

Python (con `pythontex`). Permite **correr Python** y volcar el resultado en el PDF (requiere `-shell-escape` y un paso extra `pythontex`):

```
\begin{pycode}
import numpy as np
x = np.linspace(0, 1, 5)
print(x.mean())
\end{pycode}
\py{2 + 2} % inserta 4 en el texto
```

Es ideal para que cálculos, tablas o gráficos provengan de datos reales sin copiar números a mano («documento reproducible»).

Generar y `\input`. Un patrón general: un script externo (Python, etc.) genera un `.tex` (una tabla, por ejemplo) y el documento lo incorpora con `\input{tabla_generada.tex}`. Así separás el cálculo de la presentación.

7. Compilación condicional

Producir **varias versiones** del mismo fuente (con/sin soluciones, borrador vs. final) sin duplicar el archivo:

```
% con un toggle de etoolbox (ver seccion de logica)
\newtoggle{soluciones}\toggletrue{soluciones}
... \iftoggle{soluciones}{\textbf{Sol:} 42}{ } ...

% o con el paquete comment para excluir bloques enteros:
\usepackage{comment}
\excludecomment{soluciones} % todo lo de ese entorno se omite
\begin{soluciones} ... \end{soluciones}
```

Para borrador vs. final también sirven la opción **draft** de la clase y `\includeonly` (ver *Documentos largos*).

8. Algoritmos y pseudocódigo

Para escribir **pseudocódigo** con formato (palabras clave, indentación, numeración de líneas) hay paquetes dedicados —`algorithm2e` o `algpseudocodex`—, en vez de simular a mano:

```
\begin{algorithm}
\caption{Euclides: mcd(a,b)}
\While{$b \neq 0$}{
  $t \gets b$;
  $b \gets a \bmod b$;
  $a \gets t$;
}
\Return $a$;
\end{algorithm}
```

(Para **mostrar** código fuente con resaltado —no pseudocódigo— se usan `listings` o `minted`; ver *Estándares*.)

9. Buenas prácticas

- `\newcommand` (no `\def`) para tus macros: avisa si pisás algo.
- Macros **semánticas** y centralizadas (DRY); preámbulo reutilizable.
- Lógica legible con `ifthen/etoolbox`; `\foreach` para repetir.
- `expl3` solo si desarrollás paquetes; para escribir, alcanza lo básico.
- Datos reales → `pythontex/Lua` o generá `.tex` y `\input` (reproducibilidad).

Ver también

- *Estándares* — `\newcommand` vs. `\def`, mostrar código.
- *Documentos largos* — `\includeonly`, opción **draft**.
- *Flujo e instalación* — `-shell-escape` para `pythontex/minted`.
- Manuales: `texdoc usrguide` (LaTeX), `texdoc expl3`, `texdoc pythontex`, `texdoc etoolbox`.